

08 — Understanding Python classes with an experiment record / 08 — 用实验记录理解 Python 类

The question / 问题

What is a class? How are a class, an object, and an instance related? What do `self`, `__init__`, attributes, and methods actually do in research code?

什么是类 (class) ? 类、对象 (object) 和实例 (instance) 有什么关系? 在科研代码里, `self`、`__init__`、属性 (attribute) 和方法 (method) 到底分别做什么?

This chapter builds a small record for one experiment run. Its measurements are invented teaching data, not actual experimental results. Review [03 — Python data types](#) if lists and dictionaries are unfamiliar. [05 — Dunder methods](#) explains special methods after the class basics make sense.

本章为一次实验运行创建一个小型记录。测量值是人为编写的教学数据, 不是真实实验结果。如果还不熟悉列表 (list) 和字典 (dictionary) , 请先复习 [03 — Python 数据类型](#)。理解类的基础后, 可以通过 [05 — 双下划线方法](#) 继续学习特殊方法。

Run the complete example with Python 3.9+; it needs no external packages. Read sections 1–6 first, then run the example and inspect it in Spyder. Inheritance near the end is optional.

使用 Python 3.9 或更高版本运行完整示例, 不需要外部软件包。建议先阅读第 1–6 节, 再运行示例并在 Spyder 中观察变量。靠近结尾的继承 (inheritance) 部分可以选读。

1. Start with data and an operation / 1. 从数据和一个操作开始

Suppose one run has a name and several measurements in milliseconds:

假设一次实验运行有一个名称, 以及若干以毫秒 (ms) 为单位的测量值:

```
name = "run A"
measurements = [10.0, 12.0]
mean_value = sum(measurements) / len(measurements)
```

This is already useful code. With many runs, you may want each name and measurement list to stay together, along with operations for adding a measurement and calculating its mean. A class lets us define that kind of record and its operations once, then create several records.

这段代码已经可以完成计算。当实验运行次数增多时, 你可能希望把每次运行的名称、测量值列表, 以及“添加测量值”“计算平均值”等操作放在一起。使用类 (class) , 只需定义一次这种记录及其操作, 就可以创建多条记录。

Term 术语	Meaning in our example 在本例中的含义
Class 类 (class)	ExperimentRun: a type we define for experiment records ExperimentRun: 我们为实验记录定义的一种类型
Object 对象 (object)	A value Python can work with; numbers, lists, and our records are all objects Python 可以处理的值; 数字、列表和我们的实验记录都是对象

Term 术语	Meaning in our example 在本例中的含义
Instance 实例 (instance)	One object of a particular class; run_a refers to an instance of ExperimentRun 属于某个类的一个具体对象; run_a 指向 ExperimentRun 的一个实例
Attribute 属性 (attribute)	A named item reached with a dot, such as run_a.name 通过点号访问的有名字的成员, 例如 run_a.name
Method 方法 (method)	An operation accessed through an object, such as run_a.mean() 通过对象访问的操作, 例如 run_a.mean()

“Object” and “instance” are not opposing categories: the same record is an object and an instance of ExperimentRun. The class itself is also an object. Here we focus on the records created by calling that class. Python’s [classes tutorial](#)^[1] introduces these relationships.

“对象”和“实例”不是互斥的分类：同一条记录既是对象，也是 ExperimentRun 的实例。类本身也是对象。本章重点讨论调用这个类后创建的记录。Python 官方类教程^[1]介绍了这些关系。

2. Define a complete small class / 2. 定义一个完整的小型类

```
class ExperimentRun:
    unit = "ms"

    def __init__(self, name, measurements):
        self.name = name
        self.measurements = list(measurements)

    def add_measurement(self, value):
        self.measurements.append(value)

    def mean(self):
        if not self.measurements:
            raise ValueError("Cannot calculate a mean without measurements.")
        return sum(self.measurements) / len(self.measurements)
```

Read it from the outside inward:

按从外到内的顺序阅读：

- 1. class ExperimentRun: defines our new type. Class names conventionally use capitalized words; this is a naming convention, not a requirement for execution.
class ExperimentRun: 定义了新的类型。类名通常采用每个单词首字母大写的写法；这是命名约定（naming convention），不是代码能够运行的必要条件。
- 2. The indented def blocks define methods. Their bodies are indented one level further, just like an ordinary function body.
缩进后的 def 代码块定义方法（method）。方法体再向内缩进一级，和普通函数的函数体一样。
- 3. __init__ sets the starting state: a name and a measurement list. self.name and self.measurements are instance attributes.

`__init__` 设置初始状态：名称和测量值列表。`self.name` 与 `self.measurements` 是实例属性（instance attributes）。

4. `add_measurement` changes that instance's list. It has no explicit return, so its result is `None`.

`add_measurement` 修改当前实例的列表。它没有显式写出 `return`，所以调用结果是 `None`。

5. `mean` returns a number without changing the list. An empty list raises a clear `ValueError`; its `mean` is not silently treated as zero.

`mean` 返回一个数值，不改变列表。列表为空时会抛出带有明确说明的 `ValueError`，不会悄悄把平均值当成零。

The input contract for this teaching class is a string name and a flat sequence of numeric measurements in milliseconds. It does not validate every input or convert units. `unit = "ms"` is a label, not an instruction that performs numerical conversion.

这个教学类约定的输入是：一个字符串名称，以及一个由数值组成、不嵌套其他序列的测量值序列，单位为毫秒。它没有逐项验证所有输入，也不会转换单位。`unit = "ms"` 只是单位标签，不会对数值执行单位换算。

3. Create an instance: what runs, and in which order? / 3. 创建实例：执行什么，顺序如何？

```
run_a = ExperimentRun("run A", [10.0, 12.0])
```

For this ordinary class, read that line as follows:

对于这个普通类，可以按以下过程理解这一行：

1. Calling `ExperimentRun(...)` begins creating an instance. Python's inherited `__new__` supplies the new object; we do not need to implement it.

调用 `ExperimentRun(...)` 开始创建实例。由继承而来的 `__new__` 提供新对象，本例不需要自己实现它。

2. Python calls `__init__` with the new object as `self`, "run A" as name, and the list as measurements.

Python 调用 `__init__`：新对象作为 `self` 传入，"run A" 作为 name 传入，列表作为 measurements 传入。

3. The assignments inside `__init__` store attributes on that object. The initialized object is returned by the class call and bound to the name `run_a`.

`__init__` 内的赋值语句把属性存到这个对象上。类调用返回初始化后的对象，并让变量名 `run_a` 指向它。

Strictly, `__init__` is an **initializer**, not the operation that creates the object. It must not return a non-`None` value; normally write no return in it. The [Python data model](#) ^[2] distinguishes it from `__new__`.

严格说来，`__init__` 是**初始化方法（initializer）**，并不负责创建对象本身。它不能返回非 `None` 的值，通常不必在其中写 `return`。[Python 数据模型文档](#) ^[2] 解释了它与 `__new__` 的区别。

Defining the class does not run these method bodies. Calling the class creates a record; calling a method later performs that operation. `ExperimentRun` refers to the class, whereas `ExperimentRun(...)` calls it.

定义类时，不会执行这些方法的方法体。调用类才会创建一条记录，之后调用方法才会执行相应操作。`ExperimentRun` 指向类本身，`ExperimentRun(...)` 则表示调用这个类。

4. Understand self, parameters, and the dot / 4. 理解 self、参数和点号

`self` means “the instance receiving this method call.” It is a conventional parameter name, not a keyword, global variable, or a second record. For our class, these calls have the same effect; execute only one if you want to add one value:

`self` 表示“正在接收这次方法调用的实例”。它是约定使用的参数名，不是关键字（keyword）、全局变量，也不是另外一条记录。对于本例的类，下面两种调用效果相同；如果只想添加一个值，执行其中一种即可：

```
run_a.add_measurement(14.0)          # usual form
# ExperimentRun.add_measurement(run_a, 14.0) # equivalent explicit form
```

Python supplies `run_a` as `self` in the first form. You supply only `14.0`, which becomes `value`. When you call `run_b.add_measurement(...)`, the same method works with `run_b` as `self`.

第一种写法中，Python 自动把 `run_a` 传给 `self`。你只需传入 `14.0`，它对应参数 `value`。当你调用 `run_b.add_measurement(...)` 时，同一个方法中的 `self` 就会指向 `run_b`。

Inside `__init__`, `name` is a local parameter; `self.name` is an attribute that remains on the instance after the method finishes. Writing only `name = name` would not store an instance attribute. The two names need not match: `self.name = label` works if the parameter is named `label`.

在 `__init__` 内，`name` 是局部参数（local parameter）；`self.name` 是实例上的属性，方法执行结束后仍保留在实例上。只写 `name = name` 不会保存实例属性。两边的名字也不必相同：如果参数名是 `label`，就可以写 `self.name = label`。

```
print(run_a.name)          # run A: read an attribute
run_a.name = "baseline"    # replace that attribute
print(run_a.measurements)  # [10.0, 12.0, 14.0]
print(run_a.mean())        # 12.0: call a method
calculate = run_a.mean      # store a bound method; no calculation yet
print(calculate())          # 12.0: now call it
```

The dot selects an attribute; parentheses call the selected method. `run_a.mean` and `run_a.mean()` are different expressions. Our attributes are public, so `run_a.measurements.append(16.0)` also changes the list. A class does not automatically prevent outside changes or ensure valid data.

点号用于选取属性，括号用于调用选中的方法。`run_a.mean` 和 `run_a.mean()` 是不同的表达式：前者取得绑定方法（bound method），后者才执行计算。本例的属性是公开的（public），所以 `run_a.measurements.append(16.0)` 也能改变列表。使用类不会自动阻止外部修改，也不会自动保证数据有效。

5. Separate instances, copies, and aliases / 5. 区分实例、复制和别名

Start this section with fresh records; it does not continue the optional changes above.

本节重新创建记录，不接着使用上面经过可选修改的记录。

```
source = [10.0, 12.0]
run_a = ExperimentRun("run A", source)
run_b = ExperimentRun("run B", source)
run_a.add_measurement(14.0)
source.append(99.0)

print(run_a.measurements)  # [10.0, 12.0, 14.0]
print(run_b.measurements)  # [10.0, 12.0]
print(source)              # [10.0, 12.0, 99.0]
print(run_a is run_b)      # False
```

`list(measurements)` creates a new outer list on every initialization. Therefore the two records and `source` have three different lists. Replacing it with `self.measurements = measurements` would share the supplied list, so changing one record could unexpectedly affect the other record and the caller.

每次初始化时，`list(measurements)` 都会创建一个新的外层列表。因此，两条记录各有一个列表，加上 `source`，总共是三个不同的列表。如果改为 `self.measurements = measurements`，就会共享传入的列表，改变一条记录可能意外影响另一条记录，以及调用者持有的原始列表。

This is a **shallow copy**. It suffices for our flat list of immutable numbers; nested mutable objects would still be shared. Class instances are not automatically independent: the way their attributes are assigned determines what is shared.

这是**浅复制 (shallow copy)**。本例的列表只包含不可变的数值，没有嵌套，浅复制已经够用；如果内部还有可变对象 (mutable objects)，这些内部对象仍会共享。不同实例之间的数据不会自动彼此独立，哪些内容共享取决于属性的赋值方式。

```
alias = run_a
alias.name = "renamed A"
print(run_a.name)          # renamed A
print(alias is run_a)      # True
```

`alias = run_a` creates another reference to the same instance, not a new record. This is the same assignment rule you learned for lists in chapter 03.

`alias = run_a` 让另一个变量名也引用同一个实例，并没有创建新记录。这和第 03 章学习列表时的赋值规则相同。

6. Instance attributes and class attributes / 6. 实例属性与类属性

Attribute 属性 (attribute)	Where we assign it 赋值位置	Intended role 用途
<code>self.name</code>	Inside <code>__init__</code> 在 <code>__init__</code> 内	This record's name 当前记录的名称
<code>self.measurements</code>	Inside <code>__init__</code> 在 <code>__init__</code> 内	This record's own list 当前记录自己的列表
<code>unit</code>	Directly in the class body 直接写在类体中	A common label, accessed as <code>ExperimentRun.unit</code> or <code>run_a.unit</code> 共用标签，通过 <code>ExperimentRun.unit</code> 或 <code>run_a.unit</code> 访问

In this simple class, reading `run_a.unit` falls back to the class attribute if the instance has no attribute called `unit`. Assigning `run_a.unit = "s"` adds an instance attribute that hides the class label for that record. Assigning `ExperimentRun.unit = "s"` changes the class label seen by instances without their own override. Neither assignment converts measurements. See Python's [class and instance variables](#) ^[3].

在这个简单类中，读取 `run_a.unit` 时，如果实例没有名为 `unit` 的属性，就会读取类属性（class attribute）。执行 `run_a.unit = "s"` 会新增实例属性，让当前记录优先使用自己的单位标签。执行 `ExperimentRun.unit = "s"` 则会改变类上的标签，尚未设置自己 `unit` 属性的实例都会读到这个新标签。这两种赋值都不会换算测量值。参见 Python 官方类变量与实例变量 ^[3]说明。

A common mistake is putting a mutable list in the class body:

一个常见错误是直接在类体中放置可变列表：

```
class BadRun:
    measurements = []      # one list shared through the class

first = BadRun()
second = BadRun()
first.measurements.append(10.0)
print(second.measurements) # [10.0]: surprising shared state
```

For one list per record, assign a fresh list to `self.measurements` in `__init__`. Also avoid a mutable default such as `def __init__(self, measurements=[]):` a default is evaluated once when the function is defined. If measurements are optional, use `measurements=None` and create a new list when it is `None`.

要让每条记录各有一个列表，应在 `__init__` 中把新列表赋给 `self.measurements`。也要避免 `def __init__(self, measurements=[]):` 这样的可变默认值（mutable default）：默认值只在定义函数时求值一次。如果测量值是可选参数，可以使用 `measurements=None`，并在参数为 `None` 时创建新列表。

7. Run the example and check the result / 7. 运行示例并核对结果

From the repository root, run:

在仓库根目录运行：

```
python examples/08_python_classes.py
```

Expected output:

预期输出：

```
run A: [10.0, 12.0, 14.0], mean = 12.0 ms
run B: [10.0, 12.0], mean = 11.0 ms
source: [10.0, 12.0, 99.0]
same instance: False
same measurement list: False
empty run: Cannot calculate a mean without measurements.
Verification passed.
```

Check the arithmetic yourself: $(10 + 12 + 14) / 3 = 12$ and $(10 + 12) / 2 = 11$. The script checks these values, list independence, and independence from the caller's list. It catches the intended empty-list error so execution can finish.

自己核对计算: $(10 + 12 + 14) / 3 = 12$, $(10 + 12) / 2 = 11$ 。脚本会检查这些数值、两个实例列表之间的独立性, 以及它们与调用者原始列表之间的独立性。脚本还会捕获预期的空列表异常, 确保程序能够执行到最后。

In Spyder, open the script and use these steps:

在 Spyder 中打开脚本, 按以下步骤观察:

1. Run cells 1–2 in order. `run_a`, `run_b`, and `source` are top-level names so they are convenient to inspect.
按顺序运行第 1–2 个代码单元 (cell)。`run_a`、`run_b` 和 `source` 都在脚本顶层定义, 便于直接查看。
2. In the console, evaluate `type(run_a)`, `isinstance(run_a, ExperimentRun)`, `vars(run_a)`, and `run_a.measurements`. Expect the class `ExperimentRun`, `True`, the instance's attribute dictionary, and `[10.0, 12.0, 14.0]`. A module prefix in the displayed type can vary with how you run the file.
在控制台分别执行 `type(run_a)`、`isinstance(run_a, ExperimentRun)`、`vars(run_a)` 和 `run_a.measurements`。应分别得到类 `ExperimentRun`、`True`、这个实例的属性字典, 以及 `[10.0, 12.0, 14.0]`。显示类型时的模块名前缀可能随文件运行方式而变化。
3. Put a breakpoint on the `return sum(...)` line marked P1, then debug the whole file. On the first stop, `self.name` is "run A" and `self.measurements` has three numbers. Continue to the next call to see "run B" and two numbers.
在标记为 P1 的 `return sum(...)` 行设置断点 (breakpoint), 然后调试整个文件。第一次暂停时, `self.name` 是 "run A", `self.measurements` 有三个数。继续运行到下一次调用, 就会看到 "run B" 和两个数。
4. Use the console if your Variable Explorer filters out custom objects. After editing and rerunning a class definition, recreate its instances too; existing instances still belong to the previous class object.
如果变量查看器 (Variable Explorer) 过滤掉了自定义对象, 就使用控制台查看。修改并重新执行类定义后, 也要重新创建实例; 原来已有的实例仍然属于修改前的那个类对象。

Spyder's [debugger guide](#)^[4] explains inspecting local variables at breakpoints. In this class, `vars(run_a)` shows instance attributes; `unit` is on the class and therefore does not appear there unless you assign an instance override.

Spyder 的调试器指南^[4]说明了如何在断点处查看局部变量。在本例中, `vars(run_a)` 显示实例属性; `unit` 存在于类上, 因此默认不会出现在这个字典中, 除非你给该实例单独设置了 `unit`。

Screenshot placeholder: Show Spyder paused at P1, with `self.name` and `self.measurements` visible in the console or Variable Explorer. Suggested file: `docs/images/08-class-self-inspection.png`. Replace this paragraph with the image when a screenshot is available.

截图占位 (Screenshot placeholder) : 展示 Spyder 在 P1 处暂停的画面, 并在控制台或变量查看器中显示 `self.name` 和 `self.measurements`。建议文件名: `docs/images/08-class-self-inspection.png`。有截图后, 用下方注释中的图片语句替换这组占位说明, 并移除注释标记。

8. When should you use a class? / 8. 什么时候适合使用类?

Situation 情形	A suitable starting point 适合的起点
Calculate a mean from a list once 只需对一个列表计算一次平均值	A function, such as <code>mean(values)</code> 函数 (function), 例如 <code>mean(values)</code>
Group a few settings or exchange simple records 组织少量设置, 或传递简单记录	A dictionary, such as <code>{"name": "A", "measurements": [10, 12]}</code> 字典 (dictionary), 例如 <code>{"name": "A", "measurements": [10, 12]}</code>

Situation 情形	A suitable starting point 适合的起点
Several records each keep state and use the same related operations 多条记录各自保存状态，并使用相同的相关操作	A class such as ExperimentRun 类（class），例如 ExperimentRun
Work with a numerical array or table 处理数值数组或表格	An existing NumPy or pandas type, when appropriate 在适合的情况下使用 NumPy 或 pandas 已有的类型

A class is not required for research code. Choose it when it makes relationships clearer, not because a longer program must be object-oriented. Functions can still do the calculations used by methods; methods do not require different mathematics.

科研代码并非必须使用类。当类能让数据与操作的关系更清楚时再使用它，程序变长不意味着必须改成面向对象（object-oriented）的写法。方法里需要的计算仍然可以交给普通函数完成；方法不会要求另一套数学计算方式。

9. Recognize inheritance and composition / 9. 认识继承与组合

Inheritance defines a specialized kind of an existing class. This optional example adds an operator name while reusing the measurement methods:

继承 (inheritance) 以已有类为基础，定义更具体的一类对象。下面是选读示例：它在复用测量相关方法的同时，增加操作人员名称：

```
class LabeledRun(ExperimentRun):
    def __init__(self, name, measurements, operator):
        super().__init__(name, measurements)
        self.operator = operator

labeled = LabeledRun("run C", [8.0, 10.0], "student")
print(labeled.mean())           # 9.0: inherited method
print(isinstance(labeled, ExperimentRun))  # True
```

For this single-inheritance example, `super().__init__(...)` initializes the base-class part of the same instance; it does not create another record. If you write a new initializer, call the base initializer when its setup is needed.

在这个单继承（single inheritance）示例中，`super().__init__(...)` 为同一个实例完成基类（base class）负责的初始化，并没有创建另一条记录。如果在子类中编写新的初始化方法，而又需要基类的初始化设置，就应调用基类的初始化方法。

Composition means keeping another object as part of your object. `ExperimentRun` already uses composition: it contains a list in `self.measurements` and delegates storage to that list. A future study record could contain a list of `ExperimentRun` objects. Start with a small class; elaborate inheritance trees are unnecessary here.

组合 (composition) 是把另一个对象作为当前对象的一部分。`ExperimentRun` 已经使用了组合：`self.measurements` 保存一个列表，具体存储工作由列表承担。以后可以创建一个研究记录，其中保存由多个 `ExperimentRun` 对象组成的列表。从小型类开始即可，本例不需要复杂的继承层级。

10. Common mistakes and a short practice check / 10. 常见错误与小练习

Symptom or code 现象或代码	Explanation and correction 解释与修正
NameError for ExperimentRun 使用 ExperimentRun 时出现 NameError	Run the class definition before using it. 使用之前先执行类定义。
An instance method is defined without self 定义实例方法时漏写 self	Python supplies the instance automatically; a missing parameter can cause an argument-count TypeError. Python 会自动传入实例；缺少对应参数可能引发参数数量不匹配的 TypeError。
name = name in the initializer 在初始化方法中写 name = name	This does not create an attribute; use self.name = name. 这不会创建属性，应使用 self.name = name。
run_a.mean gives a method object run_a.mean 得到方法对象	Add parentheses to calculate: run_a.mean(). 加上括号才执行计算：run_a.mean()。
run_a = ExperimentRun	This binds a name to the class; call ExperimentRun(...) to create a record. 这只让变量名指向类；应调用 ExperimentRun(...) 来创建记录。
__init__ returns a number or self __init__ 返回数值或 self	Remove that return value; the class call returns the instance. 移除这个返回值；实例由类调用返回。
One run unexpectedly changes another 修改一条记录时意外改变另一条记录	Check for shared lists or aliases; use the intended copy boundary. 检查是否存在共享列表或别名，并按需要确定哪些数据应当复制。
A typo such as run_a.mesaurments 出现 run_a.mesaurments 这样的拼写错误	Reading an absent attribute raises AttributeError; check spelling and initialization. 读取不存在的属性会引发 AttributeError；检查拼写和初始化过程。

1. Add `run_b.add_measurement(16.0)` to the section 5 example. Predict both means before running.
在第 5 节示例中加入 `run_b.add_measurement(16.0)`，运行前先预测两条记录各自的平均值。
2. Add a method `count(self)` that returns `len(self.measurements)`. Explain why the call is `run_a.count()` with no explicit `self`.
添加一个返回 `len(self.measurements)` 的方法 `count(self)`。解释为什么调用写成 `run_a.count()`，不需要显式传入 `self`。
3. Predict what `alias = run_a; alias.add_measurement(18.0)` changes. Does `run_b` change?
预测 `alias = run_a; alias.add_measurement(18.0)` 会改变什么。`run_b` 会改变吗？
4. Explain why an empty run raises an error instead of returning `0.0`.
解释为什么空记录应当抛出异常，而不是返回 `0.0`。

Check: exercise 1 gives a mean of 12.0 for A and $38 / 3$, approximately 12.6667, for B. Exercise 2 returns 3 for A at that point; Python supplies the instance. Exercise 3 changes A through its alias, leaving B unchanged. Exercise 4 has no observations to average; zero would incorrectly suggest an observed result.

核对：练习 1 中，A 的平均值为 12.0，B 为 $38 / 3$ ，约为 12.6667。练习 2 中，此时 A 的计数结果为 3，调用时由 Python 自动传入实例。练习 3 通过别名改变 A，B 不受影响。练习 4 中，没有任何观测值可供求平均；返回零会误导读者，以为确实观测到了平均值为零的结果。

Sources / 来源

Python's [classes tutorial](#)^[1], [object initialization reference](#)^[2], and [built-in inspection functions](#)^[5] document the language behavior. Spyder's [debugger guide](#)^[4] documents inspection while paused. The experiment record, numerical inputs, and exercises are teaching examples created for this handbook.

Python 官方类教程^[1]、对象初始化参考^[2]和内置查看函数^[5]说明了相关语言行为。Spyder 的调试器指南^[4]说明了暂停执行时如何查看变量。本章的实验记录、数值输入和练习都是为本手册编写的教学示例。

参考资料 (References)

1. <https://docs.python.org/3/tutorial/classes.html>
2. https://docs.python.org/3/reference/datamodel.html#object.__init__
3. <https://docs.python.org/3/tutorial/classes.html#class-and-instance-variables>
4. <https://docs.spyder-ide.org/current/panes/debugging.html>
5. <https://docs.python.org/3/library/functions.html#vars>